

Partie 2: Programmation Python

CHAPITRE 3

Listes-Tuples-Dictionnaires

Module : Informatique 1 : Algorithmique 1/Python



Pr. Mariya OUAISSA
E-mail : m.ouaissa@uca.ac.ma
Département : Informatique FSSM

Année Universitaire : 2023/2024

PLAN

1

Les listes

2

Les tuples

3

Les dictionnaires

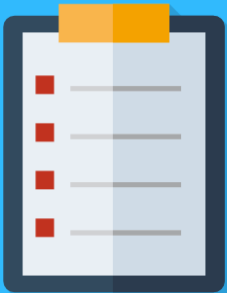


Les listes

Qu'est-ce qu'une liste et pourquoi l'utiliser ?

- Les pommes, les oranges, les poires.   
- Les chiens, les chats, les lapins.   
- Vous utilisez les listes tout le temps dans votre vie quotidienne et c'est pareil dans Python. Vous devez avoir une liste pour y enregistrer une collection d'objets auxquels vous voulez accéder plus tard.

Les listes -définition



Une liste est un tableau unidimensionnel, dont les éléments peuvent avoir des types différents (et peuvent être des listes à leur tour) ; le premier élément est identifié par l'indice 0, le dernier par l'indice -1.

Les listes - syntaxe

- Dans Python, on utilise des crochets `[]` pour indiquer une liste.
- Le code suivant crée une liste de différentes plateformes de réseaux sociaux et la sauvegarde dans une variable appelée `plateformes_sociales`.

```
1 plateformes_sociales = ["Facebook", "Instagram", "Snapchat", "Twitter"]
```

 Les listes acceptent n'importe quel type de données aussi, et même des combinaisons de types de données.

Maintenant que la liste est sauvegardée, il faut que vous puissiez accéder aux éléments à l'intérieur.

Accéder aux éléments de la liste

Pour accéder aux éléments d'une liste, on utilise un indice. Chaque élément a un indice qui lui correspond, selon sa position dans la liste. Vous obtenez la valeur de cet indice avec la syntaxe suivante : **liste[indice]**. Elle vous renverra la valeur de la liste qui est à la position de l'indice.

L'important est de noter que dans la plupart des langages de programmation, y compris Python, l'indice commence à 0, pas 1. Donc si vous voulez accéder au premier élément dans la liste ci-dessus, vous devez taper :

```
1 >> plateformes_sociales[0]
2 "Facebook"
```

Pour accéder au deuxième élément, tapez :
... et ainsi de suite.

```
1 >> plateformes_sociales[1]
2 "Instagram"
```

En particulier dans Python, vous pouvez aussi accéder aux éléments en sens inverse, en utilisant des nombres négatifs. Pour accéder au dernier élément de la liste, utilisez l'indice -1.

>> plateformes_sociales[-1]

```
1 >> plateformes_sociales[-1]
2 "Twitter"
```

Accédez aux caractères d'une chaîne comme un élément d'une liste

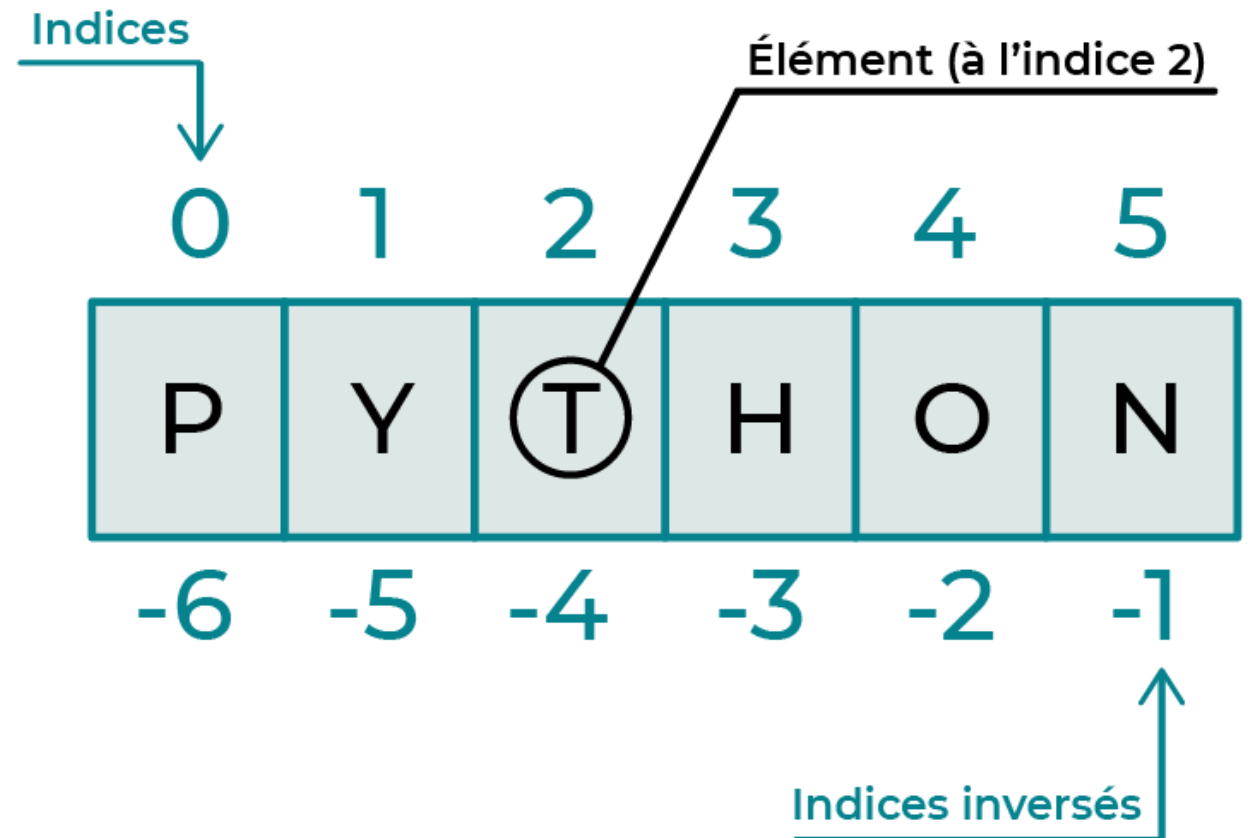
- Les indices fonctionnent aussi avec les chaînes de caractères ! En fait, les chaînes de caractères sont juste des listes de caractères. Chaque caractère correspond à un indice qui va de zéro à la longueur de la chaîne.
- Par exemple, dans la chaîne `langage = "PYTHON"` , `langage[2]` vous renverra `"T"` . Tout simplement parce que l'indice 2 dans le mot "PY**T**HON" est le "T". Ou bien, avec l'indice de la position inverse, vous devez utiliser `langage[-4]` pour accéder à `"T"` .

```
1 >> langage_de_programmation = "PYTHON"
2 >> langage_de_programmation[2]
3 "T"
4 >> langage_de_programmation[-4]
5 "T"
```

```
1 | liste          : ["girafe", "tigre", "singe", "souris"]
2 | indice positif :      0      1      2      3
3 | indice négatif :     -4     -3     -2     -1
```


Accédez aux caractères d'une chaîne comme un élément d'une liste

Pour la chaîne "Python", vous devez utiliser l'indice "2" ou l'indice inversé "-4" pour accéder au troisième caractère qui est la lettre "T".



Opérations sur les listes : Modifier –Retirer -Trier

- Tout comme les chaînes de caractères, les listes supportent l'opérateur + de concaténation, ainsi que l'opérateur * pour la duplication :

```
1 >>> an11 = ["girafe", "tigre"]
2 >>> an12 = ["singe", "souris"]
3 >>> an11 + an12
4 ['girafe', 'tigre', 'singe', 'souris']
5 >>> an11 * 3
6 ['girafe', 'tigre', 'girafe', 'tigre', 'girafe', 'tigre']
```

- Pour modifier, vous pouvez faire appel à une méthode de liste. Par exemple, pour **ajouter** une plateforme de réseau social à la fin de la liste existante, vous pouvez utiliser la méthode **append()** :

```
1 >> plateformes_sociales.append("TikTok")
2 >> print(plateformes_sociales)
3 ["Facebook", "Instagram", "Snapchat", "Twitter", "TikTok"]
```

Opérations sur les listes : Modifier –Retirer -Trier

- Pour **retirer** un élément spécifique d'une liste, vous pouvez utiliser la méthode **remove()**

```
1 >> plateformes_sociales.remove("Snapchat")
2 >> print(plateformes_sociales)
3 ["Facebook", "Instagram", "Twitter", "TikTok"]
```

- Pour connaître la longueur de la liste, utilisez la méthode **len()**.

```
1 >> len(plateformes_sociales)
2 4
```

- La dernière méthode que nous allons voir est **sort()**. Elle trie les éléments de la liste. Le tri se fait alphabétiquement pour les listes de chaînes et numériquement pour les listes de nombres.

```
1 >> plateformes_sociales.sort()
```

- **list.append(x)** : Ajoute un élément à la fin de la liste.
- **list.extend(iterable)** : Étend la liste en y ajoutant tous les éléments de l'itérable.
- **list.insert(i, x)** : Insère un élément à la position indiquée. Le premier argument est la position de l'élément avant lequel l'insertion doit s'effectuer, donc `a.insert(0, x)` insère l'élément en tête de la liste et `a.insert(len(a), x)` est équivalent à `a.append(x)`.
- **list.remove(x)** : Supprime de la liste le premier élément dont la valeur est égale à x. Une exception `ValueError` est levée s'il n'existe aucun élément avec cette valeur.
- **list.pop([i])** : Enlève de la liste l'élément situé à la position indiquée et le renvoie en valeur de retour. Si aucune position n'est spécifiée, `a.pop()` enlève et renvoie le dernier élément de la liste (les crochets autour du i dans la signature de la méthode indiquent que ce paramètre est facultatif et non que vous devez placer des crochets dans votre code ! Vous retrouverez cette notation fréquemment dans le Guide de Référence de la Bibliothèque Python).
- **list.clear()** : Supprime tous les éléments de la liste. Équivalent à `del a[:]`.
- **list.index(x[, start[, end]])** : Renvoie la position du premier élément de la liste dont la valeur égale x (en commençant à compter les positions à partir de zéro). Une exception `ValueError` est levée si aucun élément n'est trouvé. Les arguments optionnels `start` et `end` sont interprétés de la même manière que dans la notation des tranches et sont utilisés pour limiter la recherche à une sous-séquence particulière. L'indice renvoyé est calculé relativement au début de la séquence complète et non relativement à `start`.
- **list.count(x)** : Renvoie le nombre d'éléments ayant la valeur x dans la liste.
- **list.sort(*, key=None, reverse=False)** : Ordonne les éléments dans la liste
- **list.reverse()** : Inverse l'ordre des éléments dans la liste.
- **list.copy()** : Renvoie une copie superficielle de la liste.
- **list.set()** : pour supprimer les doublons dans une liste

**Extrait de la
documentation
Python**

Exercice

```
liste_nombres = [1, 6, 98, 52, 1045, 3]
```

1. Classez liste_nombres du plus petit au plus grand avec la méthode sort !
2. Supprimez le premier élément de la liste grâce à la méthode pop. P.S.: pour supprimer un élément à un index donné, on écrit liste.pop(index)
3. Ajoutez le chiffre 1097 à la liste liste_nombres grâce à la méthode de liste append !
4. Récupérez le deuxième élément de la liste liste_nombres dans la variable deuxieme_element
5. Affichez en une ligne la longueur de la liste liste_nombres après avoir supprimé le premier élément. Vous devriez obtenir 6 logiquement ! P.S.: pour calculer la longueur d'une liste, on écrit len(liste)

Correction

```
main.py x +
1 liste_nombres = [1, 6, 98, 52, 1045, 3]
2
3 # 1) classez la liste
4 liste_nombres.sort()
5 print(liste_nombres)
6 # 2) supprimez le premier élément de la liste
7 liste_nombres.pop(0)
8 # 3) ajoutez le nombre "1097" à la fin de la liste
9 liste_nombres.append(1097)
10 # 4) récupérez le deuxième élément dans une variable "deuxieme_element"
11 deuxieme_element = liste_nombres[1]
12 print(deuxieme_element) # la console devrait afficher "6" !
13
14 # 5) affichez la longueur de la liste
15 print(len(liste_nombres))
16
```

En résumé

Une liste est un moyen d'enregistrer plusieurs données ensemble.

Accédez aux éléments de la liste via l'indice, qui commence à 0.

Vous pouvez ajouter, retirer, trier les listes et bien plus en utilisant les méthodes de listes.

Les listes [] : mutable -> modifiable càd : on peut rajouter/supprimer des éléments de la liste ou les modifier

Les tuples () : immutable -> non modifiable

Les Tuples

Les tuples

- Les tuples sont une autre structure de Python qui sert à enregistrer des données.
- Au lieu des crochets `[]`, ils se caractérisent par les parenthèses `()`.

```
plateformes_sociales_tuple = ("Facebook", "Instagram", "TikTok", "Twitter")
```

- Beaucoup des propriétés des tuples sont similaires à celles des listes normales.
- Par exemple, les listes et tuples utilisent tous deux les indices. La principale différence est que les tuples sont immuables (ne peuvent pas être modifiés), alors que les listes sont modifiables.

** Les tuples sont immuables, ce qui signifie que vous ne pouvez pas modifier leurs éléments (vous ne pouvez pas ajouter de tuples, modifier ou supprimer des éléments de tuples).*

Création d'un tuple & types de données que peut contenir un tuple

- **Str, int, float et bool => tous les types de données et on peut les mélanger**
 - **Tuple1 = (1,2,5,6)**
 - **Tuple2 = ('a', "b", 'c', "d")**
 - **Tuple3 = () #empty tuple**
 - **Tuple4 = 5,3,1**
 - **Tuple5 = ("London", "Tokyo", "Korea", True, False, 1986, 1640, 1948)**

Tuples

- Les tuples sont des collections de données ordonnées et immuables (immuables). Ils peuvent être considérés comme des listes immuables. Ils sont écrits entre parenthèses :

```
>>> myTuple = (1, 2, True, "a string", (3, 4), [5, 6], None)
>>> print(myTuple)
(1, 2, True, 'a string', (3, 4), [5, 6], None)
>>> myList = [1, 2, True, "a string", (3, 4), [5, 6], None]
>>> print(myList)
[1, 2, True, 'a string', (3, 4), [5, 6], None]
```

- Chaque élément de tuple peut être d'un type différent (c'est-à-dire des entiers, des chaînes, des booléens, etc.). De plus, les tuples peuvent contenir d'autres tuples ou listes (et inversement).

Tuples

- Vous pouvez créer un tuple vide comme ceci :

```
emptyTuple = ()  
print(type(emptyTuple))      # outputs: <class 'tuple'>
```

- Un tuple à un élément peut être créé comme suit :

```
oneElemTup1 = ("one", )      # brackets and a comma  
oneElemTup2 = "one",         # no brackets, just a comma
```

- Si vous supprimez la virgule, vous direz à Python de créer une variable, pas un tuple :

-

```
myTup1 = 1,  
print(type(myTup1))          # outputs: <class 'tuple'>  
myTup2 = 1  
print(type(myTup2))          # outputs: <class 'int'>
```

Tuples

- Vous pouvez accéder aux éléments de tuple en les indexant :

```
myTuple = (1, 2.0, "string", [3, 4], (5, ), True)
print(myTuple[3])      # outputs: [3, 4]
```

- Les tuples sont immuables, ce qui signifie que vous ne pouvez pas modifier leurs éléments (vous ne pouvez pas ajouter de tuples, modifier ou supprimer des éléments de tuples).
- Cependant, vous pouvez supprimer un tuple dans son ensemble :

```
myTuple = 1, 2, 3,
del myTuple
print(myTuple)      # NameError: name 'myTuple' is not defined
```

Accès aux éléments par index de position

```
cars = ("Ferrari", "BMW", "Audi", "Jaguar")  
print(cars)  
print(cars[0])  
print(cars[1])  
print(cars[2])  
print(cars[3])
```

- **Extraction des éléments :**

With a special syntax, Python can unpack the values of a tuple and **extract** them into **single variables**.

- Let's see this with an example,

```
a,b,c = (1,2,3)
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

Tuples

- Vous pouvez parcourir les éléments d'un tuple en utilisant la boucle for:

```
t1 = (1, 2, 3)
for elem in t1:
    print(elem)
```



1
2
3

Tuples

- Vous pouvez vérifier si un élément spécifique est (ou n'est pas) présent dans un tuple en utilisant les opérateurs **in** et **not in**:

```
t2 = (1, 2, 3, 4)  
print(5 in t2)  
print(5 not in t2)|
```



```
False  
True
```


Tuples

- Vous pouvez utiliser la fonction **len()** pour vérifier combien d'éléments il y a dans un tuple:

```
t3 = (1, 2, 3, 5)  
print(len(t3))
```



4

Tuples

- Vous pouvez même joindre/multiplier des tuples

```
t1 = (1, 2, 3)
t2 = (1, 2, 3, 4)
t3 = (1, 2, 3, 5)
t4 = t1 + t2
t5 = t3 * 2
print(t4)
print(t5)
```



```
(1, 2, 3, 1, 2, 3, 4)
(1, 2, 3, 5, 1, 2, 3, 5)
```

Tuples

- Vous pouvez également créer un tuple à l'aide d'une fonction intégrée Python appelée **tuple()**.
- Ceci est particulièrement utile lorsque vous souhaitez convertir un certain itérable (par exemple, une liste, une plage (range()), une chaîne, etc.) en un tuple :

```
myTup = tuple((1, 2, "string"))
print(myTup)

lst = [2, 4, 6]
print(lst)          # outputs: [2, 4, 6]
print(type(lst))    # outputs: <class 'list'>
tup = tuple(lst)
print(tup)          # outputs: (2, 4, 6)
print(type(tup))    # outputs: <class 'tuple'>
```

Tuples

- De la même manière, lorsque vous souhaitez convertir un itérable en liste, vous pouvez utiliser une fonction intégrée Python appelée `list()`:

```
tup = 1, 2, 3,  
lst = list(tup)  
print(type(lst))           # outputs: <class 'list'>
```

Python Tuple Functions



1. len()

The len() function returns us the **length** of a **sequence** or a **container**. It is applicable on **tuples** also. Ex : len((1,2,3,4,5))

2. max()

The max() function returns us the **maximum element** from a **sequence** or **container**. The elements must contain **all numbers** or **all characters**.

Ex: max((120,20,240))

3. min()

The min() function returns the **minimum value** from group of **elements**. Ex: min((-10,-5, 3,20))

4. sum()

The sum() function calculates the **total sum** of all elements and returns it. It can only be used on **numeric values**. Ex: sum((5,10,15,20))

5. tuple()

The tuple() function **converts** an **iterable** like a **list**, **set**, **string** into an **immutable tuple**. Ex : tuple([1,2,3,4])

6. sorted()

The sorted() function sorts the elements in ascending order. It takes an iterable like a tuple and returns a **list** of **sorted elements**. Ex: sorted((4,3,2,1))

Tuples: Exercices

Exercice 1

Que se passe-t-il lorsque vous essayez d'exécuter l'extrait suivant ?

```
myTup = (1, 2, 3)
print (myTup[2])
```

- Le programme affiche 3 à l'écran

Tuples: Exercices

Exercice 2

Quelle est la sortie de l'extrait suivant ?

```
tup = 1, 2, 3  
a, b, c = tup  
  
print(a * b * c)
```


Tuples: Exercices

Exercice 2

Quelle est la sortie de l'extrait suivant ?

```
tup = 1, 2, 3
a, b, c = tup

print(a * b * c)
```

- Le programme affiche 6 à l'écran. Les éléments du tuple `tup` ont été "décompressés" dans les variables `a`, `b`, et `c`.

Tuples: Exercices

Exercice 3

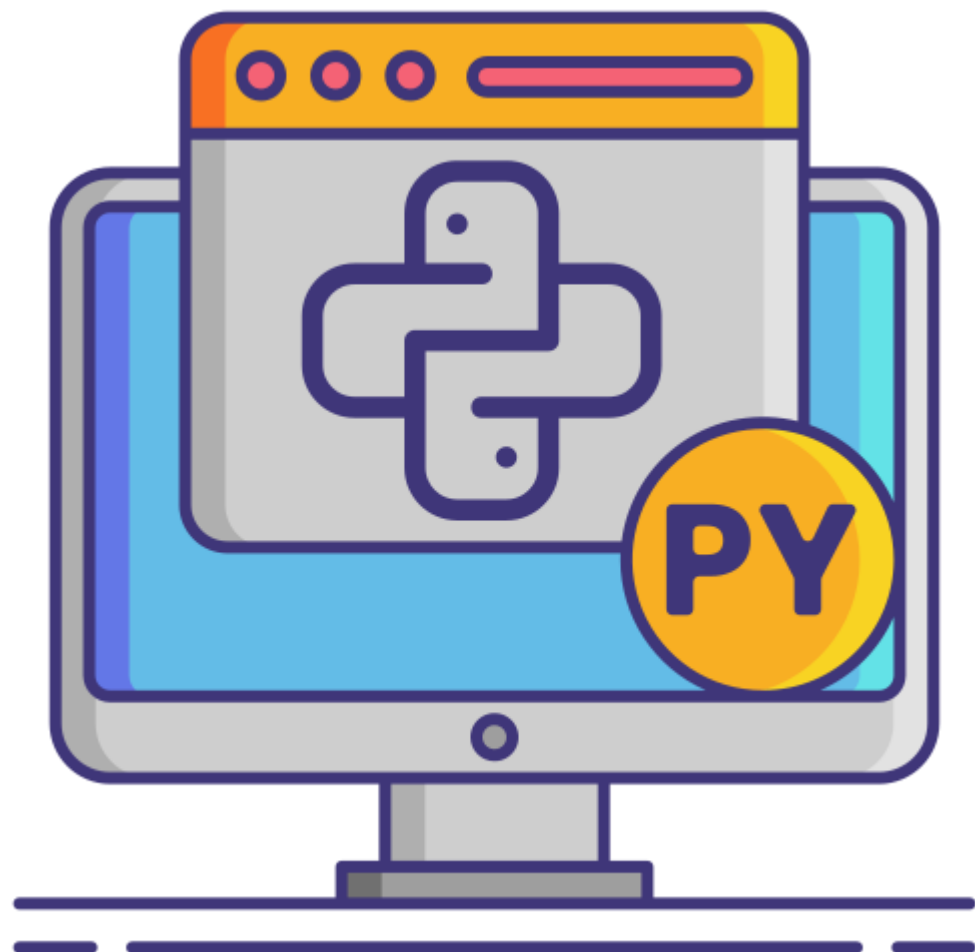
Complétez le code pour utiliser correctement la `count()` méthode pour trouver le nombre de doublons de `2` dans le tuple suivant.

```
tup = 1, 2, 3, 2, 4, 5, 6, 2, 7, 2, 8, 9
duplicates = # your code

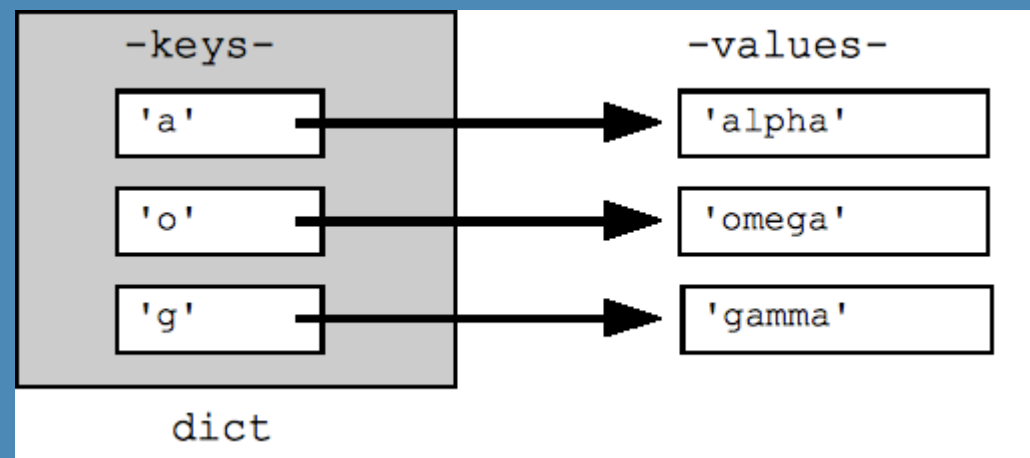
print(duplicates)    # outputs: 4
```

```
tup = 1, 2, 3, 2, 4, 5, 6, 2, 7, 2, 8, 9
duplicates = tup.count(2)

print(duplicates)    # outputs: 4
```



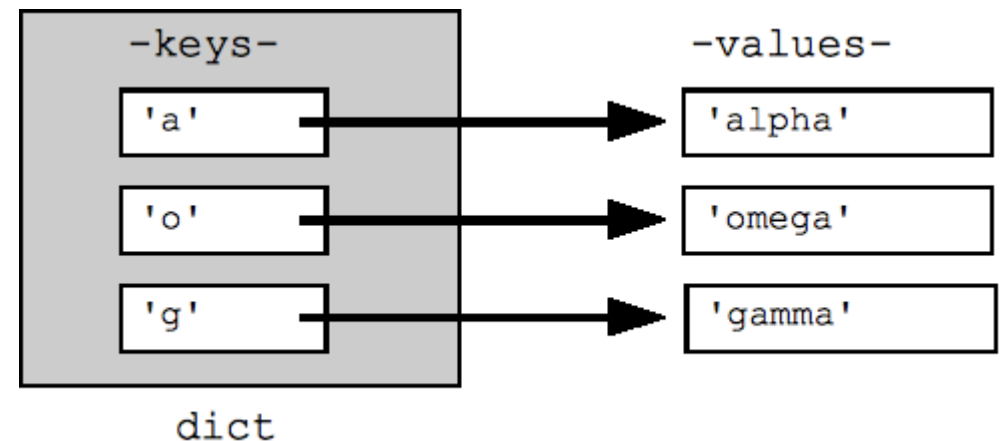
Les Dictionnaires



Dictionnaires

- Les dictionnaires sont des collections de données non ordonnées *, modifiables (mutables) et indexées. (* Dans Python 3.6x, les dictionnaires sont classés par défaut.
- Un dictionnaire en python est une sorte de liste mais au lieu d'utiliser des index, on utilise des clés, c'est à dire des valeurs autres que numériques.
- Chaque dictionnaire est un ensemble de paires **clé : valeur** . Vous pouvez le créer en utilisant la syntaxe suivante :

```
myDictionary = {  
    key1 : value1,  
    key2 : value2,  
    key3 : value3,  
}
```



Dictionnaires

- Pour initialiser un dictionnaire, on utilise la syntaxe suivante :

```
>>> a = {}
```

- Pour ajouter des valeurs à un dictionnaire il faut indiquer une clé ainsi qu'une valeur :

```
>>> a = {}  
>>> a["nom"] = "Alaoui"  
>>> a["prenom"] = "Mohamed"  
>>> a  
{'nom': 'Alaoui', 'prenom': 'Mohamed'}
```

- Vous pouvez utiliser des clés numériques comme dans la logique des listes.

Dictionnaires

- La méthode **get** vous permet de récupérer une valeur dans un dictionnaire et si la clé est introuvable, vous pouvez donner une valeur à retourner par défaut :

```
>>> data = {"nom": "Mohamed", "age": 30}
>>> data.get("nom")
'Mohamed'
>>> data.get("adresse", "Adresse inconnue")
'Adresse inconnue'
```

- Il est possible de supprimer une entrée en indiquant sa clé, comme pour les listes:

```
>>> del data["nom"]
>>> data
{'age': 30}
```

Dictionnaires

- Pour récupérer les clés on utilise la méthode **keys**

```
>>> fiche = {"nom": "Alaoui", "prenom": "Mohamed"}
>>> for cle in fiche.keys():
...     print(cle)
nom
prenom
```

- Pour récupérer les valeurs par une boucle, on utilise la méthode **values**

```
>>> fiche = {"nom": "Alaoui", "prenom": "Mohamed"}
>>> for valeur in fiche.values():
...     print(valeur)
...
Alaoui
Mohamed
```

Dictionnaires

- Pour récupérer les clés et les valeurs en même temps, on utilise la méthode **items** qui retourne un tuple.

```
>>> fiche = {"nom": "Alaoui", "prenom": "Mohamed"}
>>> for cle, valeur in fiche.items():
...     print(cle, valeur)
...
nom Alaoui
prenom Mohamed
```

Une des forces de python est la combinaison tuple/dictionnaire qui fait des merveilles dans certains cas comme lors de l'utilisation de coordonnées.

```
>>> b = {}
>>> b[(3,2)]=12
>>> b[(4,5)]=13
>>> b
{(4, 5): 13, (3, 2): 12}
```